wallarm

The Complete Kubernetes Security Guide

Often overlooked essentials to building protection and safeguarding applications and microservices.

This guide will take you through the (not-so) basics of setting up well-known areas of protection for Kubernetes security—and beyond. We will explore seven points we have identified as absolutely essential to build a secure Kubernetes environment.

- Namespaces
- RBAC
- Network Policies
- Pod Security Policies
- Audit Policy
- Credentials Management Policy
- Application Security

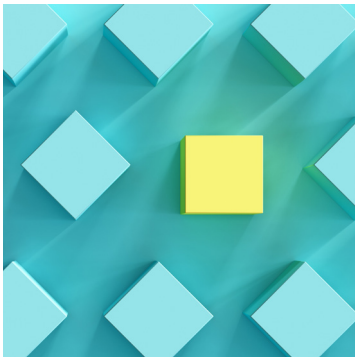
Since applications, APIs, and microservices are often vulnerable to cyber-attackers, we are going to close those entry points. You'll learn how to secure and build a security environment in Kubernetes and gain understanding of the potential ways attackers can exploit issues and distill data.

Our seventh identified area, application security, is commonly overlooked or underplayed in building security. Overconfidence in K8s and other container technologies comes from mistaking these for an extra layer of security because they provide isolation. Unfortunately, the Isolation is not complete nor does isolation equates to security. Kubernetes deployment alone can leave critical vulnerabilities.

No one can fully predict the total vulnerabilities of new, application-heavy cloud-native environments. We can make you ready to take them on.

wallarm

Content and section briefs



1

Data vs. Isolation

Data is at the root of everything

Data domains are dangerous. Data isolation is broken

Classify your data

2

The seven steps to building a secure K8s enviroment

1. Namespaces

2. RBAC—Role based access control

3. Network policies

4. Pod security policies

5. Audit policy (+ incident response)

6. Credentials management policy Application security

3

Additional resources and reference on Kubernetes AppSec

wallarm

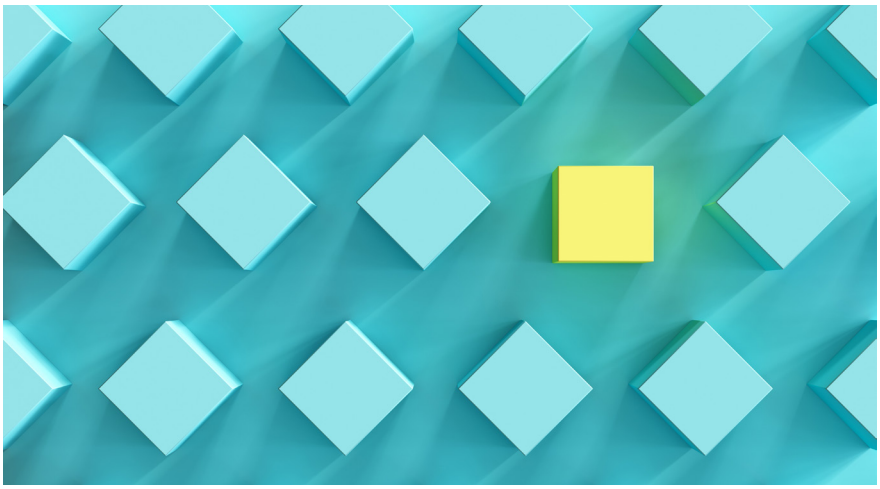
Data vs. Isolation

01. Data is at the root of everything

It's basic, but true: data is everything in protecting yourself. Say goodbye to access control lists and other "rethinking" that attempts to get around a data-centric approach. For many, the current trends in security are actually dangerously out-of-synch.

The vulnerabilities wrought by the lack of data segmentation resulted from well-intentioned efforts to build a better mousetrap. "Starting with data" was common practice years ago. As systems developed, people began relying on implementation aspects for security and partitioning. Then things changed. Take the example of network access control lists or other types of isolation efforts.

The 777 problem is a well-known example of simple security bypasses (and catastrophes) based on data partitioning or lack of thereof. Linux uses a file control mechanism to control access and provide basic security protocols. Because Linux is like windows for tech aficionados, users enjoy the command line environment. That's why users didn't throw their hands up when they frequently bumped into a problem where they were told they "do not have permission to upload file to the folder." If not constrained by the security and robustness, it is easy enough to do a list command. You can see all the files and all the access controls. And, better yet, you can change the access by . simply changing the file permission to 777 from 775. That's it. The magic number 777 was born. A few numbers in a command line. This is super dangerous because a random user can delete files. Since then, attempting to use access control lists have gotten more complex, but so has the environment.



/*

We need to build our environment based on how data actually behaves.

Unfortunately, applications and cloud computing have altered our ability to really sanitize and control a system through access control. The attempt to make an end-run around data-first approaches was undermined by the evolution of technologies and processes we simply cannot do without today.

We can use the example of someone running an online business, like a pet supply store, to illustrate security needs. For our imaginary business, Pet Toys, there are varying security needs within their application. Those differences are driven by the fact that data and entities within an environment are different. So this single pet store has things like information

about the dogs and owners, product catalogs, financial transactions, and commercial content like discounts, promotional sales, and store credits. Then you have other separate data sets, like analytics on abandoned shopping carts, customer behavior, etc. Each data type has different security requirements, which is challenging for a security professional. If a person logs on to buy a dog bowl, they actually access more than one type of data set (like product catalog). So the trick is in figuring out the security protocols that can account for the various types of dynamic data interactions.

The original data-centered approach is more closely related to and centralizes functions like a microservices approach than later access control approaches based on IP addresses and vLANs. Service is kind of the function and data is behind that function. We need to characterize everything by data before we can answer how to process the data by different functions. While this concept is basic, it is commonly overlooked.

02. Data domains are dangerous. Data isolation is broken

In theory, we could split everything by data domains or data kinds (like personal data or order data). But, in building a real app, no one can safely rely on or build around an imagined and perfect sort of data organization.

When solving a real business problem, like wanting to sell a pet toy, we are frequently presented with cases that include things like memcache rated stores. Caching technologies designed to make our operation faster mean that the partitioned data exist for some time in a combined unified form in some sort of temporary storage. This means the data isolation is broken. Additionally, there are service artifacts with data fragments, such as temporary storage and files, log file, journals, local caches, and more complexities.
No one can, realistically and completely, isolate data by type and ownership. We need to build our environment based on how data actually behaves. Without a system that adjusts for actual data interactions, the isolation provided by the data control system would be virtually useless. Another reason we can't rely on domain-isolated data is the presence of a front end service. That means if your application is exploited from the client side your data would be compromised. Under the guise of your legitimate user, access to all the website data is possible while technically disguised as legitimate activity and connected to things like log-in data. Only in an ideal world all the logical datasets can be completely isolated. So we need partition everything within its own context.

03. Classify your data

Classifying data is fundamental to everything in this guide because you are going to build on that foundation. Think of it like concrete: the wrong mixture can lead to crumbling, especially when inclimate weather (or attacks) strike.

Before you begin securing your business environment, you need to understand what you're protecting. You are not protecting an environment, application, servers, to reduce your own liability (At least, it's not th e main reason). The primary driver for AppSec is that you want to protect data—your own, your customers' data, their customers data, and down the line. That is at the heart of real protection. That includes everything that contains data, like orders, last and first names, emails or a host of other things. Ultimately, it doesn't matter the particular kind of data.

/* What Would *Google* do?

When thinking about a build, think WWGD. Google may not be a tech messiah, strictly speaking. But Google did a really good job adding extensive annotations and insights as to how to build logs in the right way.

The first mission-critical thing to do is figure out what kind of data you want to operate with before you try to isolate, secure, or change data. So, you want to survey data types.

Example

Pet Toys online shop has some
data like this:

- User data (emails, passwords, orders, shipping info, DoBs, etc.)
- Pet data (kind, breed, DoB, etc.)
- Carts (current state of orders)
- Catalog items and their prices
- Marketing elements, like banners, clicks, and sales

The seven steps to building a secure K8s enviroment

01. Namespaces

No one will guarantee your namespaces except you

Correct implementation of namespaces is important for security and many other things. Namespaces in Kubernetes are a well-known concept. The idea underlying namespaces is to divide data faster into a few virtual clusters.

Partitioning with namespaces is similar in concept to virtual hosts that are implemented on a single hosting provider. Namespaces implement the same idea for Kubernetes clusters—how to split cloud data into virtual clusters.

Here is what the official GCP Kubernetes docs have to say on the subject:

K8s does not provide a mechanism to enforce security across Namespaces. You should only use it within trusted domains and not use when you need to be able to provide guarantees that a user of the cluster or pods be unable to access any of the other Namespaces resources

Because Kubernetes does not provide a mechanism to enforce security across namespaces, you are entirely responsible for that security.

To be safe, start by only using namespaces within trusted domains. Proper namespace implementation will provide guarantees that the user of the cluster or port is unable to access any other namespaces or resources. You can apply the namespaces to implement other isolation measures in network security. That's why starting with namespace implementation is an important high-level partitioning measure. Secure namespaces before any other measure such as role-based network sport or anything similar.

Drawbacks of namespaces to keep an eye on

After their primacy, the notably unique (and potentially bad) characteristic of namespaces is that they ultimately function as a kind of list. This may feel deeply frustrating. No matter how much you want to, you cannot create a hierarchy inside namespaces. You cannot organize one namespace as a child of another namespace.

You may wish to, but you cannot create a hierarchy of namespaces. Namespaces cannot be nested within one another

GCP

On the flip side, there are some well-known tricks to circumvent that. These are implemented widely by those building production-ready environments based on Kubernetes.

Namespaces can be cleverly employed to create hierarchies

Natively, namespaces do not support structures or hierarchies. However, you can trick the system into having a hierarchy by using a naming convention with prefixes. You can define our own schema for how to build and define namespaces in your own way.

Once you have defined it, your new schema can be used to include various attributes into namespaces. Attributes could include things like the team name and environmental variables. Even more interesting is to follow a data-centric approach, as we suggested previously.

How to create an ABAC in Kubenetes using namespaces

You can also put both a data type and a team name into the namespace prefix by separating the two with a dash or some other special character. No hierarchy per se, but you can use prefixes, like:

<team>-<env>.

Keep track of and control misidentfying and misusing namespaces that have been defined (kubectl context may help).

We suggest that, at a minimum, the following schema is used:

<data-domain>-<environment>, i.e. persdata-staging, or pci-dss-prod

You can also add other attributes.
For example, the convention may include the data-domain as a first attribute and the environment classification as a second attribute after a dash. Thus, a namespace name that processes personal data in a staging environment might have a prefix like persdata-staging; a namespace which processes PCI DSS related data in production may have a prefix like

pcidss-prod.

This approach can help you later if you decide to build out based on that, applying access controls and different policies based on those data do-

/*

Start your security with namespaces because no one will guarantee them for you

Starting with namespace implementation is an important high-level partitioning measure.

main definitions. This method ensures you are creating attributes safely. It adds data as an attribute and the environment name as another attribute. You can then build out, adding more attributes in the same way.

This layered build-out is how you can implement an Attribute Based Access Control (ABAC) in Kubernetes. This schema build can serve as a basis to apply any attribute-based mechanism. Next, we'll discuss what exactly can be further built based on your custom-built ABAC. (Hint, it's RBAC.)

In terms of application security, Kubernetes is simply another kind of framework on which you can build almost anything. Starting with enforcing namespace policies, we need to continue to build security out, securing everything that follows.

02. Role based accesscontrol (RBAC)

In the previous section, you learned to implement an Attribute Based Access Control (ABAC) using prefixes in namespaces. As the Google Cloud Platform team says:

ABAC, is a powerful concept. However, as implemented in Kubernetes, ABAC is difficult to manage and understand. It requires ssh and root filesystem access on the master VM of the cluster to make authorization policy changes. For permission changes to take effect the cluster API server must be restarted.

Here is a simpler, role-based model to go forward with. We are going to propose a system with both attributes in namespaces and Role Based Access Control (RBAC). The ABAC-related approach in the section about namespaces can be supplemented with role-based models to work inside a Kubernetes environment.

What you did before within namespaces was technically an attributes-based approach. In this section, we are going to explain a role-based model instead of an attribute-based model. A role-based method avoids the problems that arise for ABAC in Kubernetes environments, as described in the previous GCP quote.

In the previous section, you created some types of attributes and namespaces. We can use that methodology in an updated way to apply attributes in RBAC. Our methodology specifically bypasses what usually renders ABAC useless in Kubernetes.

Two core ideas power RBAC: Whitelists and ClusterRoles

Two mainstays of employing RBAC are whitelisting and cluster role. Whitelisting only allows expressly permitted access. ClusterRole grants access based on a role and cluster combination of role and cluster.

RBACs are like an invite only club

All RBAC models are fundamentally whitelists only. They're like VIP lists. Rather than names, they work with categories that you can control. Imagine you're running a very exclusive club with high-profile data. This club is IT only. The private mezzanine is CISO only, where the really fancy data hangs out. Everyone who can enter is on a pre-selected list of ITs and CISOs, who qualify both because they have both the relevant role and have been added to the appropriate list of roles. Everyone IT can get into the club. Only the CISOs can access the mezzanine. Everything unlisted will be overtly denied access by default. You can vary the levels of access based on these assignations and the area being protected.

/*

RBAC can help modernize ABAC and namespaces for K8s

Whitelist only

everything unlisted denied by defaul

- **Subject**
  developer, devops, process, etc.
- **Resource**
  pod, service, etc.
- **Operation**
  action or REST method

ClusterRole

is the same of Role, plus it allows you to give permissions for:

- Non-namespaced resources, like nodes
- Resources in all the name spaces of a cluster (please avoid this)
- Non-resource (built-in) end points, like /healthz

Access is based on three things: suject, resource, and operation.

- **Subject** includes the person, place, or thing that has been whitelisted. For example a developer, DevOps, a team member, user, or process.
- **Resource** is the kind of object
For example pod, service, the cluster itself, or another logic instance related to Kubernetes.

Operation that are whitelisted are kinds of action we permit the system to do. It's an action related to REST method. For example modifications

So, following our analogy, IT and CISO roles would be the subjects in respective VIP access lists. The resource would be the club and/or the mezzanine depending on the role's whitelisted access. And the operations would be the things people are allowed to do, or club rules for mixing with data.

ClusterRoles can bring namespaces into play.

Another central consideration in understanding RBAC is the ClusterRole. The ClusterRole is an additional object. Everything related to your access controls in RBAC is based on two main objects: the role plus a cluster.

We described roles. A cluster is the same as role but also allows you to give permissions for additional things, like non-namespace resources and nodes (using namespaces covers everything to nodes, so you can avoid doing that if doing namespaces). It can include everything made with Kubernetes resources and all the namespaces of a cluster.

Note: Never allow one person to control all the namespaces. If access is set up with a wild card, it can create issues of unintended access. For example, if a newly created namespace doesn't have attributes, anybody who matches the wild card definition would have access to that. You only want to list that wanted namespace you are creating and avoid everything else. Avoid non-resource things, like built-in endpoints, which are useful for DevOps as an operation guide.

The hack for how to implement RBAC in Kubernetes

There is a hack to implement RBAC: built on attribute-based controls using the role-base concept of Kubernetes by applying namespaces.

For example, you can define a security context and the namespace, as explained earlier. This could be something like,

```
persdata-prod
```

Then, similarly define a user role like,

```
persdata-prod-reader
```

That enables this particular user to read personal data at production namespace. That way we can implement role- and context-specific privileges.

```
contexts:
- context:
    cluster: my-cluster
    namespace: persdata-prod
    user: persdata-prod-reader
  name: persdata-prod
current-context: persdata-prod
```

In this case, the organization will be easily understandable at-a-glance, across levels. It will also be obviously related to a specific context.

Note

Never allow one person to deal with all the namespaces. If access is set up with a wild card, it can create issues of unintended access. For example, if a newly created namespace doesn't have attributes, anybody who matches the wild card definition would have access to that. You only want to list that wanted namespace you are creating and avoid everything else. Avoid non-resource things, like built-in endpoints, which are useful for DevOps as an operation guide.

/*

Never allow one person to deal with all the namespaces. If access is set up with a wild card, it can create issues of unintended access.

Remember

Kubernetes does not have any default roles. That means that to take advantage of role-based controls you should define your own roles and Kubernetes will take care of enforcement. Kubernetes cannot do out of the box without your own work added.

Adding security, you can use the same context as a basis to apply different access controls—like network security or port security.

The main takeaway from this simplified example is that you can add as many attributes as you want to explain namespaces better and build this attribute-based access controls better no matter how complex the scenario

03. Network policies

The next issue, influencing Kubernetes security is Network Policies. Let's return to the GCP Kubernetes bible:

By default, pods are non-isolated; they accept traffic from any source.

This is pretty straightforward, but important enough that we decided to draw attention to it by sharing a quote. Because the pods are non-isolated and indiscriminate, without a network policy the traffic from all sources would be processed.

Once there is any NetworkPolicy in a namespace selecting a particular pod, that pod will reject any connections that are not allowed by any NetworkPolicy. (Other pods in the namespace that are not selected by any NetworkPolicy will continue to accept all traffic.)

Simple rule determines that a pod will automatically reject any expressly prohibited traffic you whitelist through a network policy. As soon as you add a NetworkPolicy declaration, Kubernetes will do the rest of the job for you, rejecting everything that is not whitelisted. While that makes this step simpler, it also means that parts of namespaces that are not included into the network policy will continue to accept traffic from all sources.
In order to make this default network policy work effectively, you have to consistently apply this network policy so that nothing is unaccounted for. If you miss something when setting network policy, that part will accept any connection, which is, of course, a big security hole that can be exploited by hackers.

How to ensure everything is 100% covered by network policy

To guarantee everything gets covered by network policy, you need tools to enforce it. One way to do that is to apply filters to your yaml files config files that can check coverage and look for holes.

A few things to remember about ensuring network policy on Kubernetes

- Use the right tools to check that you've added network policy throughout all namespaces
- Disable legacy API to retrieve metadata (Azure, GCP, AWS). You also need to disable etcd access from all the pods to prevent local exploitation through server-side issues, like a Server Side Request Forgery (SSRF) (Shopify case).
- Restrict access to K8s services like dashboard, control panels, and others (Tesla case)

Despite Kubernetes taking the world by storm, there remains many AppSec weaknesses. Attackers have numerous ways of sending requests that are particular to this environment.
To safely use Kubernetes, you have to anticipate malicious requests and payloads and disable access. That's not only coverage; it entails actively disabling access to metadata and restricting access to services. It's easy to overlook restricting access to Kubernetes services like data boot and management panels.
While in many cases these services are necessary and are run by Azure, GCP, or AWS to ensure reliability and monitor the infrastructure to prevent

/*

Without a network policy the traffic from all sources would be processed

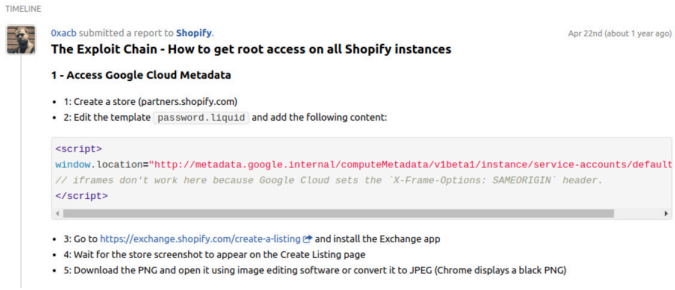
/*

Anticipate malicious requests and payloads and disable access

blackouts, exploitations, or server-side issues. However, without additional controls, these same service could become backdoors open to hackers to enable remote code execution, data compromises and other nasty business. When network policies aren't properly configured, the consequences can be devastating. The following are two examples of cyberattacks that resulted from network policy oversights that were exploited by criminals.

2 cyberattacks caused by network policy issues

The Shopify breach has become infamous. Discovered by a bug bounty hunter from HackerOne, a single API endpoint intended to fetch data for sales presentations ended up leaking revenue data and exposing thousands of files. The flaw was discovered incidentally by the whitehat hacker, Ayoub Fathi. The full extent of the data breach is unknown.



TIMELINE

0xacb submitted a report to Shopify. Apr 22nd (about 1 year ago)

The Exploit Chain - How to get root access on all Shopify instances

1 - Access Google Cloud Metadata

- 1: Create a store (partners.shopify.com)
- 2: Edit the template `password.liquid` and add the following content:

```
<script>
window.location="http://metadata.google.internal/computeMetadata/v1beta1/instance/service-accounts/default
// iframes don't work here because Google Cloud sets the `X-Frame-Options: SAMEORIGIN` header.
</script>
```

- 3: Go to https://exchange.shopify.com/create-a-listing and install the Exchange app
- 4: Wait for the store screenshot to appear on the Create Listing page
- 5: Download the PNG and open it using image editing software or convert it to JPEG (Chrome displays a black PNG)

Report submitted by bug bounty hunter, Ayoub Fathi.

The background:
Shopify is an ecommerce business that helps online retailers establish and run shops in the cloud. Businesses use Shopify services to streamline implementation of payments and storefronts. Think of it like an online mall. Businesses bring in their own merchandise and decorations. The mall owner (aka, the hosting provider) does all the maintenance for both the tenant shops in the mall and the public spaces. Moreover, that owner is providing elements of the business operations, like the new electronic checkout machines. They aren't the bank, but they are helping customers find, enter, and make purchases.

How did the SSRF attack go down?
In the Shopify incident, the hacker created his own Shopify store. Once in, the hacker simply modified the template and added some JavaScript to open a new, server-side window. That maliciously crafted file was rendered at server-side, not on the browser. (Critical factor in a Server Side Request Forgery.) It transferred the API metadata to the server-side file to access data from Shopify's Kubernetes cluster. Once the hacker had that data, he used it to steal the API keys and credentials packets from the cluster itself.

```
$ kubectl --certificate-authority ca.crt --server https://[redacted] --token "[redacted]" exec -i

Defaulting container name to web.
Use 'kubectl describe pod/[redacted] -n [redacted]' to see all of the containers in this pod.
root@[redacted]:/# id
uid=0(root) gid=0(root) groups=0(root)
root@[redacted]:/# ls
app  boot   dev  exec  key  lib64  mnt  proc  run   srv  start  tmp  var
bin  build  etc  home  lib  media  opt  root  sbin  ssl  sys    usr
root@[redacted]:/# exit
```

Huge thanks to Luís Maia 0xfad0, for helping me build this [redacted]

Impact

CRITICAL

The hacker selected the **Server-Side Request Forgery (SSRF)** weakness. This vulnerability type requires contextual information from the hacker. They provided the following answers:

01. Shopify Breach

Exploited Weakness:
API configuration flaw.

Type of attack:
SSRF Attack whereby metadata used to steal API keys and credential packets

Effect:
Thousands of stores and store-clients information was exposed

Using those keys and credentials, the hacker compromised the entire cluster. Thousands of Shopify stores (and their clients) were exposed, including revenue data, due to an API configuration flaw that let the hacker in. The hacker was able to run any code with root privileges, at every single instance, just by using that SSRF attack.

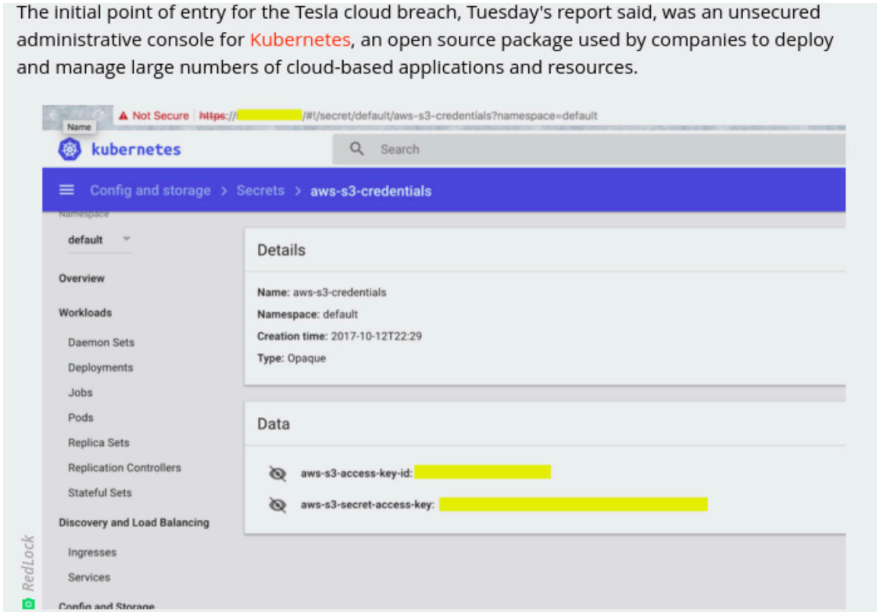
02. Tesla breach

Exploited Weakness:
Kubernetes instance and an insecure administrative

Type of attack:
alse credentials

Effect:
The total scope of the breach is yet unknown

In the Tesla breach, all the bad guys needed to know was how to Google the right application server-side issue, like a SSRF. Although it was a Tesla disgruntled employee, anyone could have found credentials and done the same. The attack, in this case, happened by way of a malicious insider. Where an attack comes from never softens the blows—stakeholders, customers, and your lost data are now insecure. You can never put the genie back in the bottle. So the greater lesson comes from asking how the employee was able to roam freely in the Tesla's data fields.
The answer boils down to insufficient administrative oversight and protocols around network policies and access. An accessible Tesla Kubernetes instance was leveraged to log in using false credentials. The entire breach boiled down to It an insecure administrative console.



This simple weakness meant that one person could single-handedly compromise the entire test infrastructure and a lot of related data. We still don't know what exactly was compromised. Potentially, all the data, including the tracking and GPS information from every single Tesla car—even your car seat operations and setting—could be compromised.
This issue could have been avoided by updating very old-fashioned network security access control methods and adding to Kubernetes configurations and defaults.

04. POD security policies

Even if you are already familiar with pod security policies, there are new insights relating to application security.

The standard approach to pod security means you have to understand and account for a long laundry list of complicated things: SELinux; AppArmor; File System; Docker; sysctl blacklist; seccom; and add-ons. It's extraordinarily difficult to get your head around everything that may fall under the roof of pod security policy. The chance for missed elements can lead to unnecessary security holes that go unnoticed. Instead, here are four (less confusing) things that are specifically made for Kubernete- by Google.

/*

Hackers look for you to take shortcuts

wallarm

Four methods can simplify and strengthen pod security:

- Privileged containers
- Privilege escalation
- Safe routing configurations
- Read-only file access

Privileged containers

Technically, a privileged container is a Docker-related way of ensuring a level of application security. By setting the right conditions in a privileged container, a service running inside the pod will not be able to escape out into the host and related devices and network.

Luckily, this very useful property of the Docker is enabled by default.

Privilege escalation

The simple privilege container configuration rule (above) can prevent local privilege escalation that would allow local containers to compromise the host.

Set AllowPriviledgeEscalation to false. Privilege escalation means that a child process can take more privileges than the parent. For example, you cannot use utilities with sticky bits from unprivileged users.

Disabling escalation protects against post-exploitation most of the time. Plus, it allows you to exclusively use privilege as you get it from your apps. Use routing to control essay limits, grammar files, system things, and the blacklist for these controls. Together, this yields an enormous AppSec deliverable.

As a bonus, you get additional protections for the network and other devices that can be used inside the containers.

Safe routing

Another important configuration option is safe routing, which uses a simple route.

A big plus of safe routing is that a temporary file system needs to be created to write files. That has to run as untrusted (unprivileged) process. The way this routing system plays out is like a game of truth or jail. Everything inside a pod should be launched as an non-route making it all usable. Building your infrastructure based on that, you can deliver apps that bolster exploitation avoidance.

Read-only file access configuration

The simplicity of read-only file access configuration shouldn't downplay the elegance. It allows a bit of customization. It allows you to protect your apps and pods by using a simple read-only file access. No changes welcome. It's like having to look at everything through museum cases. That way attackers cannot insert shells or any other active malicious content at that location. (Remember the 777 problem from earlier?)

Don't dump your sandbox for Google Cloud

Unfortunately, the above practices only apply to features implemented by Google.

That does not mean you need to dump your sandbox for Google Cloud. We recommend using all four means of Kubernetes protection. If you have Docker and like routing, go with both. They aren't mutually exclusive. SELinux provides a good illustration of multiple approaches.
Do not get overconfident or look for shortcuts. Hackers look for you to take shortcuts, too. It's not uncommon to feel that now that you're using containers, "I don't have to care about my apps to the same degree, using SELinux or other similar tools for sandbox isolation."
That thinking can leave you vulnerable. Instead, opt to keep existing sandboxes and let other solutions play, too. Add the new things, like controlling policies to implement additional features.

A read-only root file system demonstrates the four general techniques that are useful for securing any environment:

Privileged

false ← to restrict container to get an access to network and devices at the host (docker thing)

Read Only Root File system

true ← read-only FS (remember chroot/jail?)

Must Run As Non-Root

true ← chroot/jail-like thing

Allow Privilege Escalation

false ← child process can't take more privileges than parent (setuid binaries like ping will be disabled)

wallarm

04. Audit policy (+ Incident response)

Don't believe the hype: audit policy cannot actually prevent a data breach. Nonetheless, audit policy is one of the most useful inclusions in your AppSec for Kubernetes environments. Technically, audit policy is already set up by default. The way the audit policy is applied for AppSec is more relevant to incident response.

No matter what your context, you want to configure your audit policy. Do not rely on defaults.

Audit policy can help you understand a data breach before attackers are able to do significant damage or download non-trivial amount of data. What can stop breaches in real time is a combination of using audit policies in with good prevention tools.

There are two different areas under the umbrella of audit policy: stages and levels.

Stages

Request Received

The stage for events generated as soon as the audit handler receives the request.

Response Started

Once the response headers are sent, but before the response body is sent. This stage is only relevant for long-running requests (e.g. watch).

Response Complete

Once the response body has been completed

Panic

Events generated when a panic occurred

Levels

None

Don't log the events that match this rule

Metadata

Log request metadata (requesting user, timestamp, resource, verb, etc.) but not request or response body

Request

Tog event metadata and request body but not response body

Request response

Log event metadata, request and response bodies

What do stages mean in audit policy?

Stages indicate at what "stage" Kubernetes-facing requests for any particular event will be triggered. You can select a stage to request a receipt: at the point you have received a request; the response has started; the system has begun to mark the response as complete; and all the way to when the response body has been completed.

What do audit policy levels mean?

The other part of the configuration is to set up different levels— including what is known as a "noop" event. For example don't log events that match a certain name or regular expression, metadata, and local log metadata request. Other levels may include logging but not blocking anything or only blocking events with metadata and the request body, but select not to block the response body and request response.

Ultimately, the different levels mean you can log everything. Moreover, it's really wise to go ahead with that sort of comprehensive logging.
Even if you understand what's happened in an incident, store at least a couple of bases in request response away and everything else as a metadata request. That way, you can configure different timings with different log levels. This provides the framework and materials for understanding your specific risks and dealing with errors/incidents in a shorter time period. In addition, the detailed data inside those discoveries and resolutions may be invaluable down the line.
A "panic event" is generated when panic actually occurs. It is related to various errors, which can go as high as 500 responses.

/*

No matter what your context, you want to configure your audit policy. Do not rely on defaults.

wallarm

Logging is critical for your audit policies and ongoing security health.

When thinking about a build, think WWGD (What Would Google do)? Google may not be a tech messiah, strictly speaking. But Google did a really good job adding extensive annotations and insights as to how to build logs in the right way. Use the same methodology to log your own APIs and application log.

Get a comprehensive blocking system easier by applying Google best practices to your own APIs and logs. Since your development team is already doing that work (hopefully, paying attention to those annotations and insights), it's efficient and not too big a deal to ask developers to create logs based on your custom business logic logs—or any other logs you want to track and keep the same way. In doing so, you can now easily track, correlate, and manage the log systems together.

Try this pro-exercise to strengthen the quality of your product and tighten teams:

Try to be as close to the original Kubernetes logs for a month as you can in your apps. The best practice for AppSec is to make your application/ API logs 100% compatible to the same log format.

You can synch the stages and levels.

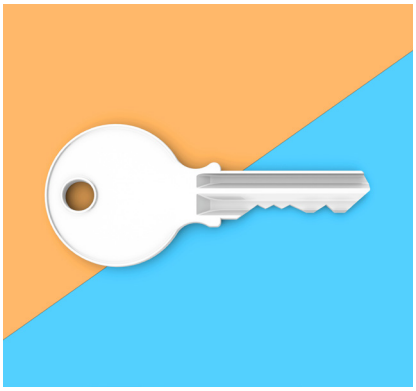
```
{
  "kind":"Event",
  "apiVersion":"audit.k8s.io/v1beta1",
  "metadata":{ "creationTimestamp":"2018-03-21T21:47:07Z" },
  "level":"Metadata",
  "timestamp":"2018-03-21T21:47:07Z",
  "auditID":"20ac14d3-1214-42b8-af3c-31454f6d7dfb",
  "stage":"RequestReceived",
  "requestURI":"/api/v1/namespaces/default/persistentvolumeclaims",
  "verb":"list",
  "user": {
    "username":"benjamin.visser@example.org",
    "groups":[ "system:authenticated" ]
  },
  "sourceIPs":[ "172.20.66.233" ],
  "objectRef": {
    "resource":"persistentvolumeclaims",
    "namespace":"default",
    "apiVersion":"v1"
  },
  "requestReceivedTimestamp":"2018-03-21T21:47:07.603214Z",
  "stageTimestamp":"2018-03-21T21:47:07.603214Z"
}
```

Remember, Kubernetes is just a web app to which you can apply API rules, including applying the same model of audit policy and log form. This consistency will also strengthen your cybersecurity landscape.

Another pro tip:

Log everything you can. If you lack sufficient storage, try optimizing by configuring different time periods. Store request response logs for a relatively shorter period. Store request metadata for longer periods, like a year. Whenever possible, store everything in full.
It's best to avoid custom scripts and use a general deployment model. That way every time you deploy a new pod, or take an action, everything will be automatically logged. Sometimes, like in deploying a custom CICD, this won't be enough and you'll have to log things yourself. Even though necessary sometimes, the implementation of this logging can get messy.

05. Credentials management policy



You are flying solo when it comes to your Credentials Management Policy. Kubernetes provides a lot of helpful capabilities, but it doesn't touch credentials. Instead, you'll need to use plugins and rely on external management of openID, certificates or secrets.
Our recommendation is certificate-only authentication policies. That way, you only need to issue a certificate for every single needed operation like inserting dates or rotating the certificates.
There are three essential advantages you can secure with your credentials management policy, as follows:
Public key infrastructure management tools
Certificates over tokens
Customization

Public key infrastructure (PKI) management tools simplify management

First, we can use any public key infrastructure management tools and products to manage that certificate, which includes all session information. You can use those tools to connect to your IDM. Everything there is already solved.
One alternative to certificates is authentication with tokens. Token management at the present level of technology doesn't quite work, though,because somebody can just copy the token or forget that a token was stored or backed up to GitHub. Token authentication will likely improve from a security standpoint and it will happen in the future. Right now certificate authentication and using public key infrastructure is a better choice.

Choose certificates over tokens

While they seem old-fashioned, certificates offer tremendous benefits that haven't been replaced by other methods. Certificates are the only means to control a specified period of time because you cannot normally issue a certificate without expiration dates. Even in instances where you technically could issue a certificate without an expiration date, adding one guarantees the certificate validity to the end point.. This is much stronger than just a plaintext token or secret.

Customized security simply fits better

If you were a ship, you'd want to make sure you had the right ammo for all the deck guns before the pirates attack. Being prepared doesn't mean finding a bunch of disparate parts that are all "emergency certified". It's about ensuring that the strategy, weapons, and training of your total defenses work together.

This is sort of an umbrella benefit—certificates allow you greater customization and flexibility. For example, you can use object I.D.s for additional attribute-based features as leverage. You can apply additional logic that way and, again, connect it to your IDM. Plus, using certificates is easier than tokens. Unlike with tokens, you can copy and paste the certificate contents of any private key contents certificate and public information. Plus certificates store better than other approaches.
When configuring a credentials management policy choose one with highly customizable, tried-and-true approach for Kubernetes security

Why credentials management adds effectiveness to security

The above three benefits work well together and are not limited in their scope. An enormous benefit to using credentials is they can be applied both to infrastructure credentials and all your services. This unification simplifies security and can dramatically improve oversight. Simpler systems that are better understood also encourage a lot more actual adoption and continued adherence.

You can see the benefit of unified methodologies when things break from the ordinary. For example, one day you may have to integrate a custom API of a third-party provider who has no certificate authentication, but who you have to use as frequently as possible. Having a unified creden tial-based system can help keep you protected.

Another boost for security comes in choosing a platform with native encryption support. A few years ago, Kubernetes introduced a new feature called Native Encryption Support. You can encrypt both tokens as they are stored in BTCB and all other data. Despite Native Encryption being gen-erally useful, it's not an ideal method to secure tokens, since, even when encrypted, the tokens would be stored on the same storage volumes as other data.

06. Application security

So far, we have been talking about protecting Kubernetes environments, but haven't said anything directly about Application Security. Yet, all of the aforementioned protective measures are relevant to application security. You cannot truly protect a modern cloud environment, like Kubernetes, without considering applications.

Attacks can come from multiple directions. To start our work on applica-tion security for Kubernetes, we have to cognitively separate application security—or application specific vulnerabilities, in particular—into two parts. On one hand, you have service-side application specific vulnera-bilities. Service-side vulnerabilities can help provide hackers access to reports, as we saw earlier in the Shopify breach. On the other, you have client-side elements that can help attackers gain access to user data.

Once we've mentally separated the two attack fronts, we can move into explaining how each of those attack fronts are already and can further be protected.

AppSec follows from the first six security methods

If you did everything right in points one through six—namespace through credentials management—you are ready to lock down your Kubernetes environment. At this point, you can definitely mitigate a breach in a variety of ways.

The bad news is that these combined processes are pretty detailed and complicated; you have implement everything in the right way for it to work. Attackers are opportunists, not assassins. If you miss something during implementation, it can leave a hole in the barrier big enough for an attacker to get through.

Related to the client-side, the aforementioned policies may help identify exploitation (if done right), provided that we can control every single piece of data policies apply to within our organization. In effect, that mighty control goal is related to application security itself. Even though it's not strict Kubernetes security on the surface, building security without paying attention to applications is like leaving a service tunnel unwatched at Fort Knox.

You cannot secure Kubernetes without securing applications

Through applications, malicious payloads can wriggle their way into bigger areas of contamination. Let's say you have something as benign seeming as a single page app that works with your API based on Kuber-netes. Such an application, if not properly protected, can be exploited by a single client-side issue and will be related to a part of that single page app or its JavaScript. It is unrelated to Kubernetes at this point. But, that is just the first stage of an attack if the application is not properly monitored and secured.

We'll get to some working examples a little further on. (See the Python and API Gateway problems.) The point is, with applications working with your API, based on Kubernetes, it is a great conduit to attacks and issues that are significantly larger than they appear at first blush.

/*

Attackers are op-portunists, not as-sassins. If you miss something dur-ing implementa-tion, it can leave a hole in the barri-er big enough for an attacker to get through.

/*

Even microservices inside your infra-structure should not automatically be assumed to be trust-worthy.

wallarm

Direct action for securing applications for K8s

Here are a few whitehat tips to understand how to protect your applications—and reinforce the six points of protection you have just examined. First, always use privileged ports, which means that somebody else cannot open ports from zero to 1003 without the right privileges.

Second, disable

net.ipv4.tcp_fastopen

inside Kubernetes. Fastopen is a relatively new mechanism introduced in the 2.6kernel about seven years ago to make connections between microservices faster. Now, this is default. It helps attackers sometimes to still attack from one IP address to another IP address and then keep that packet and use that same token that time when the attacker-compromised port will be disabled or will change an IP address.

Third, follow the list of other hacker-insight-based tips below. Run everything from a non-root account. Never use host-based authentication. Never try to define access controls using host names, IP addresses, or other network parameters that are unreliable in the dynamic Kubernetes environments. Instead, use attributes that authenticate everything appropriately, as described and discussed earlier.

Fourth, the right tools also help. Check the code with static source analysis and dynamic security testing tools, implementing API protection solutions and complementary API security processes.

Security testing and monitoring helps guarantee you actually know when and where a threat started, how to mitigate the threats, and, generally, how to monitor in increasingly better ways.

Attack scenarios drive home the case for application security

The two following attacks are from the real world, though we can not disclose the names of the companies. They both relate to security audits and show how attackers work.

Case 1: Python attacks can allow hackers to hunt big

In our first example, auditors found a remote code execution in a major tech company's python app because of a disorientation bug.

- RCE in a Python app because of the Pickle deserialization
- Python (tornado) web service was running on an unprivileged port 5001
- Self-kill + port re-open from other process to sniff data
- Stolen East-West secret was used to steal the data

The developers in this scenario had just published a service that contained personally identifiable data. (Technically, the data about people was actually code packaged and presented in a UI to make it readable for humans.) By doing that, the data was put up as a binary tree. Attackers used that code to sneak malicious data in around additional connected services. They soon discovered that the initial service ran on an unprivileged port.

Once they had their malicious code out in the world, then they did a sort of self-kill of that service. The hackers ran an additional web server at the same port as the initial compromised web server, running it as a pod. The port was unprivileged, allowing hackers sneak inside the infrastructure. Inside the system, the attackers were able to sit back and listen, monitoring all communication that crossed their path. Those communications gave them valuable, secret information and knowledge from those services. Those secrets hold a high value when sold on the black market.
The damage wasn't limited to the initial point of entry, or main company. Everyone connected to that service, services connected to those services, and other services and API controllers down the line were accessed and their data exposed. It's less of a domino effect and more like swine flu

Hacker insights for AppSec measures

- Always use privileged UNIX ports (0-1023) for your services to avoid overmapping by race conditions
- Disable net.ipv4.tcp_fastopen inside if you don't mind
- Run everything from non-root
- Do not use host-based authentication
- Check code for vulnerabilities by SAST and DAST both
- Implement L7 firewalls (API WAFs)

that big of a secret. If mistakes can be made with large security teams, it should humble us all and serve as a reminder to use all the layers of protection—meticulously.

What can we learn from IBM's Python Problem?

It's unfortunately common that people prefer to house all their security hopes and powers in one place, like one server, for the sake of time. But it only took a small bit of unidentified virus to start the Black Plague. (That plague strain is now known, existent, and no longer a threat. Much like passe attacks.)

The sorts of mistakes that laid siege to IBM can happen with large and small operations alike, especially with the pressure to streamline being high based on complexity for the former or the resources scarce on the latter.

Case 2: Criminals simply register for private data, and get it

There is no need to hack, when you can ask. Attack scenario number two also started as a distillation bug, but in Ruby on Rails instead of Python. Criminals simply registered for private data at the API gateway—and got it.

API Gateway Attack Scenario

1

RoR marshaller Remote Code Execution as an entry point

2

Push-based microservices publishing at API gateway (Service Registry)

3

Compromised service (let's say shipping service) registered itself as an authentication service (login+passwords stolen)

Starting in the same way as the previous attack scenario, attackers found that they could compromise the push-based publishing service at the API Gateway.

To get in, attackers simply registered their service as another temporary gateway. They were able to do that after using the compromised token to get access to the port, where they could run their code. After that, they sent a push request to API gateways to register themselves as a new service.

The hacker-created API Gateway easily added the registered service to the load balancers. Then, it began sending the login requests for that service. It was simple: one service can register itself at an API gateway as another service—such as an authentication service—and then it kicks back and receives logins and passwords that unlock related addresses, and so on. It's an easy way to pilfer user information and run with it.

/*

Criminals simply registered for private data at the API gateway—and *got it.*

The takeaway: microservice environments are very useful, but they are not safe without special measures.

This API gateway exploitation illuminates a larger problem. This sort of malicious data theft happens all the time within microservice environments.

People forget that simply because a microservice is there, doesn't mean it's valid. Even microservices inside your infrastructure should not automatically be assumed to be trustworthy. In fact, your network control cannot account for every API you rely on.

Moreover, if a microservice, API-like process, or some application code is compromised, it could render you vulnerable to large scale problems. Those compromised elements can make it possible to send any requests from that infrastructure unit to different units, including the API gateway.

Conclusion

Drones, tunnels, spies—the imagination of hackers far outpaces the production of genre espionage, modern war, and sci-fi movies. So, we need new ways of thinking about how attackers get into our system. Because our system, as it enters the cloud, is itself more nebulous and easily entered without our notice.

In summary, to truly secure your K8s environment for deployment, you have to lock down all of the following:

- Namespaces
- RBAC
- Network Policies
- Pod Security Policies
- Audit Policy
- Credentials Management Policy
- Application Security

Applications are vital for an online entity to grow quickly and provide unsurpassable usability, new features, and responsiveness to customers and internal teams alike. Nonetheless, they are also the most common attack entry points.

Building security in a Kubernetes environment is challenging, but the best practices and tools can help. Your security health will be an evolution, like all pioneering expeditions. The most important thing to remember is that the quality of what you do is as important as anything. There is mastery in good deployment and policy crafting. And, it's not one thing you have to do. Set up multiple layers of defense with special care and continual audits of how your various protection methods are working.

wallarm

Additional resources and reference on Kubernetes AppSec

Namespaces

https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/
https://kubernetes.io/docs/tasks/administer-cluster/namespaces-walkthrough/
https://kubernetes.io/blog/2016/08/kubernetes-namespaces-use-cases-insights/

RBAC

https://kubernetes.io/docs/reference/access-authn-authz/rbac/
https://kubernetes.io/blog/2017/04/rbac-support-in-kubernetes/
https://www.cncf.io/blog/2018/08/01/demystifying-rbac-in-kubernetes/
https://jeremievallee.com/2018/05/28/kubernetes-rbac-namespace-user.html
https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#config_set-context/ context details

Network Policies

https://kubernetes.io/docs/concepts/services-networking/network-policies/
https://kubernetes.io/docs/concepts/cluster-administration/networking/
https://docs.bitnami.com/kubernetes/how-to/secure-your-kubernetes-application-with-networkpolicies/
https://github.com/ahmetb/kubernetes-network-policy-recipes
https://medium.com/@reuvenharrison/an-introduction-to-kubernetes-network-policies-for-security-people-ba92dd4c809d

Pod Security Policies

https://kubernetes.io/docs/concepts/policy/pod-security-policy/
https://resources.whitesourcesoftware.com/blog-whitesource/kubernetes-pod-security-policy
https://kubernetes.io/docs/tasks/configure-pod-container/security-context/

Audit Policy

https://kubernetes.io/docs/tasks/debug-application-cluster/audit/
https://cloud.google.com/kubernetes-engine/docs/concepts/audit-policy

About this article

This whitepaper based on the research and thought leadership of Wallarm AI CEO, Ivan Novikov. Wallarm is a quickly rising application security company headquartered in San Francisco. Ivan is also a bug hunter and white hat hacker, making him a hands-on, proven expert in application security. His passion is sharing experience on application security and related aspects, like Kubernetes environments.